

Create Blob storage resources with the .NET client library

In the cloud shell toolbar, in the Settings menu, select Go to Classic version (this is required to use the code editor).

1. Create a resource group for the resources needed for this exercise. Replace myResourceGroup with a name you want to use for the resource group. You can replace eastus2 with a region near you if needed. If you already have a resource group you want to use, proceed to the next step.
2. In this Cloudslice lab, your Resource Group has already been created (myResourceGrouplod58016530) and you may safely skip this step.
3. `az group create --location eastus2 --name myResourceGroup`
4. Many of the commands require unique names and use the same parameters. Creating some variables will reduce the changes needed to the commands that create resources. Run the following commands to create the needed variables. Replace myResourceGroup with the name you're using for this exercise.
5. `resourceGroup=myResourceGrouplod58016530, location=eastus, accountName=storageacct$RANDOM`
6. Run the following commands to create the Azure Storage account, each account name must be unique. The first command creates a variable with a unique name for your storage account. Record the name of your account from the output of the echo command.

```
az storage account create --name $accountName \  
--resource-group $resourceGroup \  
--location $location \  
--sku Standard_LRS
```

```
echo $accountName
```

7. Assign a role to your Microsoft Entra user name. To allow your app to create resources and items, assign your Microsoft Entra user to the Storage Blob Data Owner role. Perform the following steps in the cloud shell.
8. Tip: Resize the cloud shell to display more information, and code, by dragging the top border. You can also use the minimize and maximize buttons to switch between the cloud shell and the main portal interface.
9. Run the following command to retrieve the userPrincipalName from your account. This represents who the role will be assigned to.

TypeCopy

```
userPrincipal=$(az rest --method GET --url https://graph.microsoft.com/v1.0/me \  
--headers 'Content-Type=application/json' \  
--output json | jq -r .userPrincipalName)
```

```
--query userPrincipalName --output tsv)
```

10. Run the following command to retrieve the resource ID of the storage account. The resource ID sets the scope for the role assignment to a specific namespace.

```
resourceID=$(az storage account show --name $accountName \
--resource-group $resourceGroup \
--query id --output tsv)
```

Run the following command to create and assign the Storage Blob Data Owner role. This role gives you the permissions to manage containers and items.

```
11. az role assignment create --assignee $userPrincipal \
--role "Storage Blob Data Owner" \
--scope $resourceID
```

12. Create a .NET console app to create containers and items

Now that the needed resources are deployed to Azure the next step is to set up the console application. The following steps are performed in the cloud shell.

Run the following commands to create a directory to contain the project and change into the project directory.

```
mkdir azstor
```

```
cd azstor
```

Create the .NET console application.

```
dotnet new console
```

Run the following commands to add the required packages in the application.

```
dotnet add package Azure.Storage.Blobs
```

```
dotnet add package Azure.Identity
```

Run the following command to create a data folder in your project.

```
mkdir data
```

Now it's time to add the code for the project.

Add the starter code for the project

13. Run the following command in the cloud shell to begin editing the application.

code Program.cs

Replace any existing contents with the following code. Be sure to review the comments in the code.

14.

```
csharp
```

```
using Azure.Storage.Blobs;
```

```
using Azure.Storage.Blobs.Models;
```

```
using Azure.Identity;
```

```
Console.WriteLine("Azure Blob Storage exercise\n");
```

```
// Create a DefaultAzureCredentialOptions object to configure the  
DefaultAzureCredential
```

```
DefaultAzureCredentialOptions options = new()
```

```
{
```

```
    ExcludeEnvironmentCredential = true,
```

```
    ExcludeManagedIdentityCredential = true
```

```
};
```

```
// Run the examples asynchronously, wait for the results before proceeding
```

```
await ProcessAsync();
```

```
Console.WriteLine("\nPress enter to exit the sample application.");
```

```
Console.ReadLine();
```

```
async Task ProcessAsync()
```

```
{
```

```
    // CREATE A BLOB STORAGE CLIENT
```

```
    // CREATE A CONTAINER
```

```
    // CREATE A LOCAL FILE FOR UPLOAD TO BLOB STORAGE
```

```
    // UPLOAD THE FILE TO BLOB STORAGE
```

```
// LIST BLOBS IN THE CONTAINER  
  
// DOWNLOAD THE BLOB TO A LOCAL FILE  
  
}
```

Press ctrl+s to save your changes, and continue to the next step.

Add code to complete the project

15. dotnet run

In the Azure portal, navigate to the Azure Storage account you created.

Expand > Data storage in the left navigation and select Containers

Select the container the application created and you can view the blob that was uploaded.

Run the two commands below to change into the data directory and list the files that were uploaded and downloaded.

```
cd data
```

```
ls
```

16. Clean up resources

Now that you finished the exercise, you should delete the cloud resources you created to avoid unnecessary resource usage.

In your browser navigate to the Azure portal <https://portal.azure.com>; signing in with your Azure credentials if prompted.

Navigate to the resource group you created and view the contents of the resources used in this exercise.

17. On the toolbar, select Delete resource group.

Enter the resource group name and confirm that you want to delete it.

CAUTION: Deleting a resource group deletes all resources contained within it. If you chose an existing resource group for this exercise, any existing resources outside the scope of this exercise will also be deleted.